Autobahn 패밀리

📋 패밀리 개요

Autobahn은 **Optimistic BFT** 계열의 합의 알고리즘으로, 정상 상황에서 **n-f threshold**를 사용하여 빠른 합의를 달성하고, 문제 발생 시 **2f+1 fallback**으로 안전성을 보장합니다.

핵심 아이디어: Optimistic Fast Path

```
┌──────────────────────────────────┐
│  정상 (Optimistic)               │
│  n-f votes (정직한 노드만)        │
│  → 빠른 합의 ✓                   │
└──────────────────────────────────┘
            ↓ (문제 발생)
┌──────────────────────────────────┐
│  문제 (Fallback)                 │
│  2f+1 votes (전통적 BFT)          │
│  → 안전한 합의 ✓                  │
└──────────────────────────────────┘
```

Threshold 비교

```
n = 4, f = 1:

Optimistic (n-f):  3 votes (75%)
Fallback (2f+1):   3 votes (75%)

n = 7, f = 2:

Optimistic (n-f):  5 votes (71%)
Fallback (2f+1):   5 votes (71%)

n = 10, f = 3:
```

```
Optimistic (n-f):  7 votes (70%)
Fallback (2f+1):   7 votes (70%)
```

주의: 숫자는 같지만 의미가 다름!

- **n-f**: "정직한 노드 모두"
- **2f+1**: "최소 f+1 정직 노드 포함"

패밀리 구성 (5가지)

1. **Autobahn (Original)**: 원본 Autobahn
2. **Autobahn-Basic**: n-f optimistic + 2f+1 fallback
3. **Autobahn-Fast**: 3-tier (n, n-f, 2f+1)
4. **Autobahn-Pipelined**: Pipeline 처리 + watermark
5. **Autobahn-Sync**: Synchronous 2-phase (n-f only)

공통 개념: Optimistic BFT

왜 n-f가 빠른가?

전통적 BFT (2f+1):

```
4 nodes: 3 votes 필요
- Node 0: ✓ (fast)
- Node 1: ✓ (fast)
- Node 2: ✓ (fast)
- Node 3: ✗ (slow - Byzantine or network delay)

→ Wait for Node 3? NO!
→ 3 votes = 2f+1 = commit ✓
→ But had to wait for slow node's timeout...
```

Optimistic BFT (n-f):

```
4 nodes: 3 votes 필요 (n-f = 4-1 = 3)
- Node 0: ✓ (fast)
- Node 1: ✓ (fast)
- Node 2: ✓ (fast)
```

```
- [Node 3: 응답 전에 이미 commit!]

→ 3 votes = n-f = commit immediately! ✓
→ 느린 노드 기다리지 않음!
```

Optimistic의 의미

가정: 대부분의 시간에 모든 정직한 노드는 빠르게 응답
최적화: 정직한 노드(n-f)만으로 합의
이득: Byzantine이나 느린 노드 기다릴 필요 없음

---

1. Autobahn (Original) & Autobahn-Basic

📋 개요

원본 Autobahn과 Basic 변형은 **2가지 모드**를 지원합니다.

2-Mode 구조

```go
type Mode int
const (
    Optimistic Mode = iota  // n-f votes
    Fallback                // 2f+1 votes
)
```

동작 과정

```
Leader              Replica1 (fast)     Replica2 (fast)     Replica3 (slow)
  |                     |                   |                   |
  |---PREPARE---------->|                   |                   |
  |---PREPARE---------->|------------------>|                   |
  |---PREPARE---------->|------------------>|------------------>|
  |                     |                   |                   |
  |<--VOTE--------------|                   |                   |
  |<--VOTE--------------|<------------------|                   |
  |<--VOTE--------------|<------------------|<--[timeout]-------|
```

```
  |                    |                  |                  |
  | Check votes:                                             |
  |   - 3/4 votes = n-f = Optimistic! ✓                     |
  |                    |                  |                  |
  |---COMMIT(opt)------>|                  |                  |
  |---COMMIT(opt)------>|------------------>|                  |
  |---COMMIT(opt)------>|------------------>|------------------>|
```

Mode 전환 로직

```go
func (node *AutobahnNode) handlePrepareVotes() {
    voteCount := len(node.prepareVotes[node.currentSeq])

    if voteCount >= node.numNodes - node.f {
        // Optimistic mode!
        node.mode = Optimistic
        node.sendCommit(Optimistic)
    } else if voteCount >= 2*node.f + 1 {
        // Fallback mode
        node.mode = Fallback
        node.sendCommit(Fallback)
    } else {
        // Not enough votes yet
        wait()
    }
}
```

2-Phase Consensus

Phase 1: Prepare

```
Leader:
  1. Block 생성
  2. PREPARE(block) 전송
  3. n-f 또는 2f+1 votes 대기

Replica:
  1. PREPARE 수신
  2. 검증
  3. VOTE → Leader
```

Phase 2: Commit

```
Leader:
  1. Mode 결정 (Optimistic or Fallback)
  2. COMMIT(block, mode) 전송

Replica:
  1. COMMIT + mode 수신
  2. Mode에 따라 처리:
     - Optimistic: 즉시 실행
     - Fallback: 추가 검증 후 실행
```

메시지 구조

```go
type PrepareMsg struct {
    sequence int
    block    *Block
    leader   int
}

type VoteMsg struct {
    sequence  int
    blockHash string
    voter     int
}

type CommitMsg struct {
    sequence  int
    blockHash string
    mode      Mode      // Optimistic or Fallback
    leader    int
}
```

코드 위치

- Original: `internal/algorithms/autobahn/autobahn.go`
- Basic: `internal/algorithms/autobahn_basic/autobahn.go`

---

2. Autobahn-Fast - 3-Tier Thresholds

📋 개요

Autobahn-Fast는 **3가지 경로**를 지원하여 네트워크 상태에 맞게 최적화합니다. Jolteon의 아이디어를 Autobahn에 적용한 버전입니다.

3-Tier 구조

```go
type PathMode int
const (
    SuperOptimistic PathMode = iota  // n votes (100%)
    Optimistic                       // n-f votes (정직한 노드)
    Fallback                         // 2f+1 votes (최소 quorum)
)
```

Threshold 계산 (n=4, f=1)

```go
superOptimisticThreshold := 4       // 100% (모든 노드)
optimisticThreshold      := 3       // 75% (n-f)
fallbackThreshold        := 3       // 75% (2f+1)
```

경로 선택 알고리즘

```go
func (node *AutobahnFastNode) selectPath(voteCount int) PathMode {
    if voteCount >= node.numNodes {
        // 모든 노드 동의!
        return SuperOptimistic
    } else if voteCount >= node.numNodes - node.f {
        // 정직한 노드 모두
        return Optimistic
    } else if voteCount >= 2*node.f + 1 {
        // 최소 quorum
        return Fallback
    }
    return NoPath
}
```

동작 시나리오

시나리오 1: 완벽한 네트워크 (Super-optimistic)

```
Leader proposes Block A
All 4 nodes vote immediately
→ 4/4 votes (n)
→ Super-optimistic path!
→ Instant commit (가장 빠름) ✓✓✓
```

시나리오 2: 한 노드 느림 (Optimistic)

```
Leader proposes Block A
3 nodes vote fast, 1 node slow
→ 3/4 votes (n-f)
→ Optimistic path!
→ Fast commit (빠름) ✓✓
```

시나리오 3: 여러 노드 느림 (Fallback)

```
Leader proposes Block A
3 nodes respond, 1 node timeout
→ 3/4 votes (2f+1)
→ Fallback path
→ 2-phase commit (안전) ✓
```

Dynamic Path Downgrade

```go
func (node *AutobahnFastNode) handleVotes() {
    for {
        vote := <-node.voteChan
        node.recordVote(vote)

        voteCount := node.getVoteCount()

        // 계속 최고 경로 확인
        newPath := node.selectPath(voteCount)

        if newPath > node.currentPath {
            // 더 나은 경로로 업그레이드!
            node.currentPath = newPath
            node.commit(newPath)
            break
        }
    }
}
```

비교: Autobahn-Fast vs Jolteon

특징	Autobahn-Fast	Jolteon
Base	Autobahn (2-phase)	HotStuff (3-phase)
경로 수	3	3
Super-opt	n votes	n votes
Opt	n-f votes	n-f votes
Fallback	2f+1 (2-phase)	2f+1 (2-phase)

코드 위치

```
internal/algorithms/autobahn_fast/autobahn_fast.go
```

---

3. Autobahn-Pipelined - Pipeline Processing

📋 개요

Autobahn-Pipelined는 **여러 블록을 동시에 처리**하여 처리량을 높입니다. Watermark 기반으로 순서를 보장합니다.

Pipeline 구조

```
┌──────────────────────────────────────────┐
│           Pipeline (max depth=3)         │
├──────────────────────────────────────────┤
│  Sequence 1: [Prepare] → [Commit] → Ready│
│  Sequence 2: [Prepare] → [Commit] → Ready│
│  Sequence 3: [Prepare] → [Commit]        │
│  Sequence 4: [Prepare]                   │
├──────────────────────────────────────────┤
│  nextCommitSeq = 1 (watermark)           │
```

```
│  → Commit 1, 2, 3 in order!              │
└──────────────────────────────────────────┘
```

PipelinedBlock 구조

```go
type PipelinedBlock struct {
    Block    *common.Block
    Sequence int
    Ready    bool  // Prepare + Commit 완료
}
```

Pipeline Map

```go
type AutobahnPipelinedNode struct {
    pipeline          map[int]*PipelinedBlock  // [seq] → block
    nextCommitSeq     int                      // watermark
    maxPipelineDepth  int                      // 3
}
```

동작 과정

```
Time ──────────────────────────────────────▶

t=0:  Seq 1: Prepare
t=1:  Seq 1: Commit, Seq 2: Prepare
t=2:  Seq 1: Ready!, Seq 2: Commit, Seq 3: Prepare
t=3:  Seq 2: Ready!, Seq 3: Commit, Seq 4: Prepare

      [tryCommitPipeline]
      → Commit Seq 1 ✓
      → Commit Seq 2 ✓
      → Wait for Seq 3...
```

Ordered Commit (Watermark)

```go
func (node *AutobahnPipelinedNode) tryCommitPipeline() {
    for {
        block := node.pipeline[node.nextCommitSeq]
```

```go
        if block == nil || !block.Ready {
            // 다음 블록이 준비 안 됨, 대기
            break
        }

        // 순서대로 커밋!
        node.commitBlock(block.Block)
        delete(node.pipeline, node.nextCommitSeq)
        node.nextCommitSeq++
    }
}
```

Pipeline Depth 제한

```go
func (node *AutobahnPipelinedNode) canPropose() bool {
    pipelineSize := len(node.pipeline)
    return pipelineSize < node.maxPipelineDepth
}

func (node *AutobahnPipelinedNode) propose() {
    if !node.canPropose() {
        // Pipeline full, wait
        return
    }

    block := node.createBlock()
    node.startPrepare(block)
}
```

예제: 3 블록 파이프라인

```
Pipeline state:

Seq 5: [Prepare done] [Commit done] Ready=true
Seq 6: [Prepare done] [Commit done] Ready=true
Seq 7: [Prepare done] [Commit ....] Ready=false
Seq 8: [Prepare ....]                Ready=false (full!)

nextCommitSeq = 5

[tryCommitPipeline called]
```

```
Step 1: Check Seq 5
  - Ready? Yes!
  - Commit Seq 5 ✓
  - nextCommitSeq = 6

Step 2: Check Seq 6
  - Ready? Yes!
  - Commit Seq 6 ✓
  - nextCommitSeq = 7

Step 3: Check Seq 7
  - Ready? No (Commit in progress)
  - Stop here, wait

[Now pipeline has space, can propose Seq 9!]
```

장점

✅ **높은 처리량**: 여러 블록 동시 처리
✅ **순서 보장**: Watermark로 순차 커밋
✅ **깊이 제한**: 메모리 관리

코드 위치

```
internal/algorithms/autobahn_pipelined/autobahn_pipelined.go
```

---

4. Autobahn-Sync - Synchronous 2-Phase

📋 개요

Autobahn-Sync는 **동기 네트워크를 가정**하여 n-f threshold만 사용합니다. Fallback이 없어 가장 간단합니다.

핵심 가정: Synchrony

```
동기 네트워크 (Synchronous Network):
- 메시지 지연 상한 있음 (예: 1초 이내 도착 보장)
```

```
- 노드 처리 시간 상한 있음
- Timeout으로 실패 감지 가능
```

vs 비동기:

```
비동기 네트워크 (Asynchronous Network):
- 메시지 지연 무한정 가능
- 실패 vs 느림 구분 불가
- 더 보수적인 quorum 필요 (2f+1)
```

단순화된 구조

```go
type AutobahnSyncNode struct {
    // Fallback 없음!
    onlyThreshold int  // n-f만 사용
}

func NewAutobahnSyncNode(n, f int) *AutobahnSyncNode {
    return &AutobahnSyncNode{
        onlyThreshold: n - f,  // 정직한 노드 수
    }
}
```

2-Phase (Optimistic Only)

```
Leader              Replica1            Replica2            Replica3
  |                     |                   |                   |
  |---PREPARE---------->|                   |                   |
  |---PREPARE---------->|------------------>|                   |
  |---PREPARE---------->|------------------>|------------------>|
  |                     |                   |                   |
  |<--VOTE--------------|                   |                   |
  |<--VOTE--------------|<------------------|                   |
  |<--VOTE--------------|<------------------|<------------------|
  |                     |                   |                   |
  | Wait for n-f (3) votes                                      |
  | → Guaranteed in sync network!                               |
  |                     |                   |                   |
  |---COMMIT----------->|                   |                   |
  |---COMMIT----------->|------------------>|                   |
  |---COMMIT----------->|------------------>|------------------>|
```

```
  |                    |                    |                    |
  | Execute!                                                      |
```

동작 로직

```go
func (node *AutobahnSyncNode) handlePreparePhase() {
    // Prepare 전송
    node.broadcast(PREPARE, node.currentBlock)

    // n-f votes 대기 (항상 도착!)
    node.waitForVotes(node.onlyThreshold)

    // Commit phase로
    node.sendCommit()
}

func (node *AutobahnSyncNode) handleCommitPhase() {
    // Commit 전송
    node.broadcast(COMMIT, node.currentBlock)

    // n-f commits 대기
    node.waitForCommits(node.onlyThreshold)

    // 실행!
    node.execute(node.currentBlock)
}
```

동기 환경 이점

안전성:

```
동기 네트워크에서 n-f 응답 = 모든 정직한 노드 응답
→ Byzantine 노드가 영향 못 줌
→ 2f+1 fallback 불필요!
```

성능:

```
항상 Optimistic path (n-f)
→ 가장 빠른 합의
→ Fallback 오버헤드 없음
```

Timeout 설정

```go
const (
    PrepareTimeout = 1 * time.Second   // 동기 가정: 1초 내 도착
    CommitTimeout  = 1 * time.Second
)

func (node *AutobahnSyncNode) waitForVotes(threshold int) {
    timer := time.After(PrepareTimeout)

    for {
        select {
        case vote := <-node.voteChan:
            node.recordVote(vote)
            if node.hasThreshold(threshold) {
                return  // 성공!
            }

        case <-timer:
            // Timeout = 실패 (동기 가정 위반)
            node.handleTimeout()
            return
        }
    }
}
```

언제 사용하나?

✅ **LAN 환경**: 지연 예측 가능
✅ **데이터센터**: 낮은 지연, 높은 신뢰성
✅ **프라이빗 블록체인**: 제어된 네트워크

❌ **인터넷**: 지연 예측 불가
❌ **WAN**: 네트워크 불안정

코드 위치

`internal/algorithms/autobahn_sync/autobahn_sync.go`

📊 Autobahn 패밀리 비교

특징	Original/Basic	Fast	Pipelined	Sync
경로 수	2 (Opt, Fall)	3 (Super, Opt, Fall)	2 (Opt, Fall)	1 (Opt only)
Threshold	n-f, 2f+1	n, n-f, 2f+1	n-f, 2f+1	n-f
병렬 처리	❌	❌	✅ (pipeline)	❌
네트워크 가정	Async	Async	Async	Sync
복잡도	중	중	높	낮
처리량	중	중	높 ✓	중
지연시간	중	낮 ✓	중	낮 ✓

🎯 핵심 개념 정리

1. Optimistic vs 2f+1

```
Optimistic (n-f):
- "정직한 노드만으로 충분"
- 빠른 응답자만 대기
- Byzantine/slow 노드 무시

Traditional (2f+1):
- "최소 f+1 정직 노드 보장"
- 모든 가능성 대비
- 보수적
```

2. 2-Phase의 효율성

```
PBFT (3-phase):
  Pre-prepare → Prepare → Commit

Autobahn (2-phase):
  Prepare → Commit
```

```
→ 33% phase 감소!
```

3. Threshold 선택

```
n=7, f=2:

Super-optimistic (n):     7 (100%) - 완벽한 경우
Optimistic (n-f):         5 (71%)  - 정상 경우 ✓
Fallback (2f+1):          5 (71%)  - 최악의 경우
```

💻 공통 구현 패턴

Quorum 확인

```go
func (node *AutobahnNode) hasQuorum(votes map[int]bool, threshold int) bool {
    return len(votes) >= threshold
}

func (node *AutobahnNode) checkAllThresholds(voteCount int) {
    if node.variant == Fast {
        // Check 3 thresholds
        if voteCount >= node.n {
            return SuperOptimistic
        } else if voteCount >= node.n - node.f {
            return Optimistic
        } else if voteCount >= 2*node.f + 1 {
            return Fallback
        }
    } else {
        // Check 2 thresholds
        if voteCount >= node.n - node.f {
            return Optimistic
        } else if voteCount >= 2*node.f + 1 {
            return Fallback
        }
    }
}
```

Mode 전환

```go
func (node *AutobahnNode) handleModeSwitch(newMode Mode) {
    if newMode < node.currentMode {
        // Downgrade (Opt → Fall)
        log.Printf("Switching to fallback mode")
        node.currentMode = newMode
    } else if newMode > node.currentMode {
        // Upgrade (Fall → Opt)
        log.Printf("Switching to optimistic mode")
        node.currentMode = newMode
    }
}
```

🔬 성능 특성

장점

✅ **Optimistic 최적화**: 정상 시 빠름
✅ **동적 적응**: 네트워크 상태에 맞춤 (Fast)
✅ **높은 처리량**: Pipeline 변형
✅ **간단함**: Sync 변형

단점

❌ **복잡한 모드 전환**: Optimistic ↔ Fallback
❌ **Synchrony 가정**: Sync 변형은 제한적
❌ **메모리**: Pipelined 변형

🎓 학습 포인트

1. **Optimistic의 핵심**:
    - n-f = 정직한 노드만
    - 느린 노드 기다리지 않음
    - 정상 시 최적
2. **Multiple Paths**:

    - 네트워크 상태에 적응
    - Fast 변형의 3-tier
    - 동적 경로 선택

3. **Pipeline**:
    - 병렬 처리로 처리량 향상
    - Watermark로 순서 보장
    - Depth 제한으로 메모리 관리

📚 참고 자료

- [Autobahn 개념] Rambaud et al. (2022) - Extending adaptivity of async BFT
- [Optimistic BFT] Fast Byzantine Consensus
- [Jolteon 논문] Gelashvili et al. (2022) - Responsive BFT
- [코드] `internal/algorithms/autobahn*/`

🔗 관련 알고리즘

- **Jolteon**: 유사한 3-tier threshold (HotStuff 기반)
- **Fast-HotStuff**: 2-tier optimistic (HotStuff 기반)
- **HotStuff**: O(n) 메시지 복잡도 기반

← Narwhal Family | Main Index →